

An Analysis of Long short-term memory

Long short-term memory

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$h_t = o_t \odot \tanh(c_t)$$

Long short-term memory -- Part 1

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f c_{t-1} + b_f) \\ i_t &= \sigma(W_i x_t + U_i c_{t-1} + b_i) \\ o_t &= \sigma(W_o x_t + U_o c_{t-1} + b_o) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \sigma(W_c x_t + b_c) \\ h_t &= o_t \odot \sigma(h(c_t)) \end{aligned}$$

Long short-term memory -- Part 2

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\ c_t &= \sigma(W_c x_t + U_c h_{t-1} + b_c) \\ \tilde{c}_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ h_t &= o_t \odot \sigma(h(\tilde{c}_t)) \end{aligned}$$

Long short-term memory -- Part 3

Applications 2001: Gers and Schmidhuber trained LSTM to learn languages unlearnable by traditional models such as Hidden Markov Models. Hochreiter et al. used LSTM for meta-learning (i.e. learning a learning algorithm). 2004: First successful application of LSTM to speech Alex Graves et al. 2005: Daan Wierstra, Faustino Gomez, and Schmidhuber trained LSTM by neuroevolution without a teacher. Mayer et al. trained LSTM to control robots. 2007: Wierstra, Foerster, Peters, and Schmidhuber trained LSTM by policy gradients for reinforcement learning without a teacher. Hochreiter, Heusesel, and Obermayr applied LSTM to protein homology detection the field of biology. 2009: Justin Bayer et al. introduced neural architecture search for LSTM. 2009: An LSTM trained by CTC won the ICDAR

connected handwriting recognition competition. Three such models were submitted by a team led by Alex Graves. One was the most accurate model in the competition and another was the fastest. This was the first time an RNN won international competitions. 2013: Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton used LSTM networks as a major component of a network that achieved a record 17.7% phoneme error rate on the classic TIMIT natural speech dataset. 2017: Researchers from Michigan State University, IBM Research, and Cornell University published a study in the Knowledge Discovery and Data Mining (KDD) conference. Their time-aware LSTM (T-LSTM) performs better on certain data sets than standard LSTM. 2020: Kaplan and McCandlish et al. explored the scaling law of LSTM architecture in language modeling and compared it with Transformers, showing that Transformers scale better than LSTM on text data.

Long short-term memory -- Part 4

2015: Google started using an LSTM trained by CTC for speech recognition on Google Voice. According to the official blog post, the new model cut transcription errors by 49%. 2016: Google started using an LSTM to suggest messages in the Allo conversation app. In the same year, Google released the Google Neural Machine Translation system for Google Translate which used LSTMs to reduce translation errors by 60%. Apple announced in its Worldwide Developers Conference that it would start using the LSTM for quicktype in the iPhone and for Siri. Amazon released Polly, which generates the voices behind Alexa, using a bidirectional LSTM for the text-to-speech technology. 2017: Facebook performed some 4.5 billion automatic translations every day using long short-term memory networks. Microsoft reported reaching 94.9% recognition accuracy on the Switchboard corpus, incorporating a vocabulary of 165,000 words. The approach used "dialog session-based long-short-term memory". 2018: OpenAI used LSTM trained by policy gradients to beat humans in the complex video game of Dota 2, and to control a human-like robot hand that manipulates physical objects with unprecedented dexterity. 2019: DeepMind used LSTM trained by policy gradients to excel at the complex video game of Starcraft II.

Long short-term memory -- Part 5

Long short-term memory (LSTM) is a type of recurrent neural network (RNN) aimed at mitigating the vanishing gradient problem commonly encountered by traditional RNNs. Its relative insensitivity to gap length is its advantage over other RNNs, hidden Markov models, and other sequence learning methods. It aims to provide a short-term memory for RNN that can last

An Analysis of Long short-term memory

Long short-term memory

thousands of timesteps (thus "long short-term memory"). The name is made in analogy with long-term memory and short-term memory and their relationship, studied by cognitive psychologists since the early 20th century. An LSTM unit is typically composed of a cell and three gates: an input gate, an output gate, and a forget gate. The cell remembers values over arbitrary time intervals, and the gates regulate the flow of information into and out of the cell. Forget gates decide what information to discard from the previous state, by mapping the previous state and the current input to a value between 0 and 1. A (rounded) value of 1 signifies retention of the information, and a value of 0 represents discarding. Input gates decide which pieces of new information to store in the current cell state, using the same system as forget gates. Output gates control which pieces of information in the current cell state to output, by assigning a value from 0 to 1 to the information, considering the previous and current states. Selectively outputting relevant information from the current state allows the LSTM network to maintain useful, long-term dependencies to make predictions, both in current and future time-steps. LSTM has wide applications in classification, data processing, time series analysis tasks, speech recognition, machine translation, speech activity detection, robot control, video games, healthcare, energy forecasting.

Long short-term memory -- Part 6

Training An RNN using LSTM units can be trained in a supervised fashion on a set of training sequences, using an optimization algorithm like gradient descent combined with backpropagation through time to compute the gradients needed during the optimization process, in order to change each weight of the LSTM network in proportion to the derivative of the error (at the output layer of the LSTM network) with respect to corresponding weight. A problem with using gradient descent for standard RNNs is that error gradients vanish exponentially quickly with the size of the time lag between important events. This is due to $\lim_{n \rightarrow \infty} W^n = 0$ $\{\displaystyle \lim_{n \rightarrow \infty} W^n = 0\}$ if the spectral radius of W $\{\displaystyle W\}$ is smaller than 1. However, with LSTM units, when error values are back-propagated from the output layer, the error remains in the LSTM unit's cell. This "error carousel" continuously feeds error back to each of the LSTM unit's gates, until they learn to cut off the value.